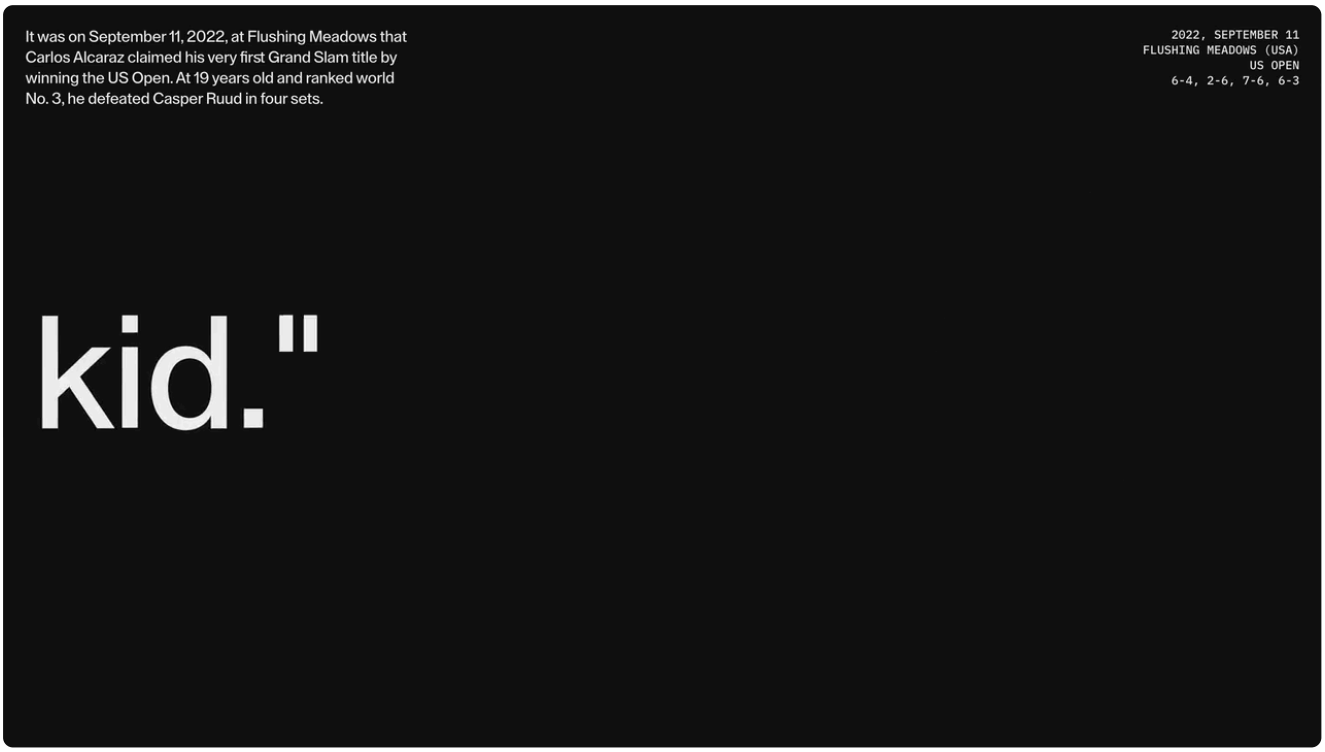
Made With Gsap

011
Smooth letters



In this effect, we will make a sentence scroll from right to left as the user scrolls. As the sentence moves, the letters will gradually position themselves correctly. It's a perfect way to capture the user's attention. By the way, do I have your attention too? Then let's go!

HTML Structure

The structure is relatively simple. The sentence inside our `container` will move from right to left while remaining visible within the viewport.

```
<section class="mwg_effect011">
    <div class="container">
        <p class="text">This is something I've dreamed of since I was a kid.</p>
    </div>
</section>
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

. hen the phrase extends beyond the viewport, we hide any overflow from the root element of our component:

```
.mwg_effect011{
    overflow: hidden;
}
```

The properties `white-space: nowrap` and `width: max-content` ensure that the sentence does not wrap and maintains its full width. This will be crucial for our upcoming calculations.

```
.mwg_effect011 .text {
    padding: 0 101vw; // To avoid seeing the sentence directly
    display: flex;
    width: max-content;
    white-space: nowrap;
}
```

To allow each letter of our sentence to undergo transformations, we set them to `display: inline-block`.

```
.mwg_effect011 .letter {
    display: inline-block;
}
```

Font loading

To ensure that the future `letter` elements are positioned correctly and that the sentence retains its proper width, I wait for the font to be fully loaded before proceeding.

```
document.fonts.ready.then(() => {
    // Once the font is loaded, let the fun begin!
})
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

function called Made With Gsap each letter and wrap it in a `<span>`. This function takes as a parameter the DOM element containing the phrase we want to split.

```
const text = document.querySelector('.mwg_effect011 .text')
wrapLettersInSpan(text)

function wrapLettersInSpan(element) {
    const text = element.textContent;
    element.innerHTML = text
        .split('')
        .map(char => char === ' ' ? '<span> </span>' : `<span class="letter">${char}</s
        .join('');
}
```

To achieve this, I use the `split()` method to break the text into an array, using each character (`''`) as a separator. The `map()` function then wraps each character in a `<span>`. One special case: if the current character is a space, we replace it with ` `, which represents a non-breaking space.

Finally, I reconstruct the phrase by placing each character back in order. The newly modified string (with its DOM) is then returned to the original element in parameter.

Moving the Sentence

Now that our sentence is made up of elements, we can move it from right to left. I start by calculating the scroll distance:

```
const distance = text.clientWidth - document.body.clientWidth
```

If I move the entire sentence, it would completely exit the screen, which is why I subtract one screen width. This ensures that the translation stops exactly when the end of the sentence reaches the right edge of the viewport. Why not use `window.innerWidth` to calculate the screen width? Because `document.body.clientWidth` takes into account the browser's scrollbar width.

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

n. Inside its `scrollTrigger` the `container` during the animation. The `scrollTrigger` object also allows me to define when the tween animation should trigger.

```
const scrollTween = gsap.to(text, {
    x: - distance, // Translation on the x-axis
    ease: 'none', // Linear movement
    scrollTrigger: {
        trigger: '.mwg_effect011 .container', // Listen to the container's position
        pin: true, // We pin the division container
        end: '+=' + distance // We unpin it at the end of the animation
        scrub: true, // Progress with scrolling
    }
});
```

Notice that the tween is stored in the variable `scrollTween`, which we will use in the next step.
To go further, we could choose a different scrolling speed in the `scrollTween` variable, for example `end: '+=' + distance / 2` to make the text scroll twice as fast compared to the default scroll speed.

Letter Movement

To apply different values to each letter, I use the native JavaScript function `forEach()`. This also allows us to track the position of each letter independently within our `scrollTrigger` objects.

```
letters.forEach( (letter, index) => {
    ...
})
```

For each letter, I create a tween using the `gsap.from()` method, which sets an initial value for the animation. I focus on two properties:

- `- yPercent`: To move each letter along the y-axis
- `- rotation`: To rotate each letter

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

`th.random() - 0.5` returns a

Made With Gsap

lying it by a chosen number increases the amplitude of the transformation.

```
letters.forEach(letter => {
    gsap.from(letter, {
        yPercent: (Math.random() - 0.5) * 400, // Between -200 & 200
        rotation: (Math.random() - 0.5) * 60, // Between -30 & 30
        ease: "elastic.out(1.2, 1)", // Will bounce at the end of the animation
        scrollTrigger:{
            trigger: letter, // Listens to the tween's letter position
            containerAnimation: scrollTween,
            start: 'left 100%',
            // Animation starts when the letter is at the right edge of the viewport
            end: 'left 0%',
            // Ends when the letter reaches the left edge of the viewport
            scrub: 0.5 // Progresses with scrolling, with a 0.5s delay
        }
    })
})
```

Remember, I stored the tween of the container in a variable earlier. Now, we specify that each letter's animation should be linked to the container's movement by setting `containerAnimation: scrollTween`. This method is recommended for animating child elements within a horizontal motion, as it allows us to use `left` and `right` values in the `start` and `end` properties.

Go further

To take this effect even further, you could experiment with different `ease` for each letter. Here, I chose `elastic.out()`, which adds a bounce at the end of the animation, but you could increase its amplitude and the number of bounces for a different feel.
To easily fine-tune these values, I recommend exploring the Easing page in the GSAP documentation.

In this example, I used Lenis smooth scroll to further enhance the fluidity and smoothness. If you'd like to apply it as well, check out our Go Further page to learn how to add it.

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

HTML CSS JAVASCRIPT

```
<section class="mwg_effect011">
    <p class="scroll">Scroll</p>
    <div class="container">
        <div class="header">
            <p>It was on September 11, 2022, at Flushing Meadows that Carlos Alcaraz c
            <p>2022, September 11 <br />Flushing meadows (USA) <br />Us Open <br />6-4
        </div>
        <p class="text">"This is something I've dreamed of since I was a kid."</p>
    </div>
</section>
```

Wanna learn something else?

BACK TO THE COLLECTION

Never miss what's next

Your email SUBMIT

By submitting your email, you'll be the first to know about upcoming updates for Made With Gsap. You can unsubscribe at any time.

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

X(TWITTER)
INSTAGRAM
BLUESKY
LINKEDIN

PAGES

HOME
COLLECTION
FAQ
PRICING

CONTACT

REACH US
SUPPORT

Made With Gsap

2025©

PERMISSIONS & TERMS

PRIVACY POLICY